Session 8 Probability Basics

Theoretical Probability, Empirical Simulations & Set-Based Joint Events

1. Fundamental Probability Function (Coin Toss Example)

Task: Create a Python function `calculate_probability` and evaluate getting 'Heads' in a fair coin toss.

```
def calculate_probability(event_count, total_outcomes): if total_outcomes == 0: return 0.0 return event_count / total_outcomes # Tossing a fair coin: 1 desired outcome ('Heads'), 2 total outcomes ('Heads', 'Tails') p_heads = calculate_probability(1, 2) print(f"Theoretical Probability of Heads: {p_heads:.2f} ({p_heads * 100:.1f}%)")
```

PROGRAM OUTPUT

Theoretical Probability of Heads: **0.50 (50.0%)**

2. Dice Roll Empirical Simulation (100 Trials)

Task: Simulate rolling a 6-sided die 100 times using `random.randint()` and calculate the experimental probability of getting '6'.

```
import random random.seed(42) # Set seed for reproducible results rolls = [random.randint(1, 6) for _ in range(100)] six_count = rolls.count(6) experimental_prob = six_count / 100 print(f"Count of '6' in 100 rolls: {six_count}") print(f"Experimental P(6): {experimental_prob:.2f} | Theoretical P(6): {1/6:.2f}")
```

SIMULATION OUTPUT

Count of '6' in 100 rolls: **17** | Experimental P(6): **0.17 (17.0%)** (Theoretical ≈ 16.67%)

3. Spotify Genre History Probability Calculation

Task: Given an array of 20 recent Spotify track genres, find the empirical probability of picking a 'pop' song.

```
recent_genres = [ 'pop', 'rock', 'pop', 'hiphop', 'jazz', 'pop', 'indie', 'pop', 'rock', 'pop', 'hiphop', 'pop', 'classical', 'pop', 'rock', 'pop', 'pop', 'jazz', 'pop', 'hiphop' ] pop_count = recent_genres.count('pop') total_songs = len(recent_genres) p_pop = pop_count / total_songs print(f"Pop Tracks: {pop_count}/{total_songs} | P(Pop) = {p_pop:.2f}")
```

PROGRAM OUTPUT

Pop Tracks: **10/20** | Probability P(Pop): **0.50 (50.0%)**

4. Joint Event Probability: Zomato Food & Dessert Orders

Task: Calculate P(Food ∩ Dessert) — the probability that a user selected at random from all unique customers ordered both food and dessert.

```
def prob_food_and_dessert(food_users, dessert_users): set_food = set(food_users) set_dessert = set(dessert_users) # Intersection (users who ordered both) both_users = set_food.intersection(set_dessert) # Union (all total unique users across both lists) all_unique_users = set_food.union(set_dessert) if not all_unique_users: return 0.0 return len(both_users) / len(all_unique_users) # Test Data food_customers = ["U1", "U2", "U3", "U4", "U5"] dessert_customers = ["U3", "U4", "U6", "U7"] p_both = prob_food_and_dessert(food_customers, dessert_customers) print(f"Joint Probability P(Food ∩ Dessert): {p_both:.4f}")
```

SET OPERATIONS OUTPUT

Both (Intersection): {U3, U4} (2) | Total Unique (Union): 7 | **P(Both) = 2/7 = 0.2857 (28.57%)**

5. Flipkart Recommendation Engine via Conditional Probability

Concept: Conditional Probability `P(Headphones | Phone) = P(Headphones ∩ Phone) / P(Phone)` determines how likely a user is to purchase headphones given they bought a smartphone.

```
def recommendation_probability(bought_phone, bought_both): if bought_phone == 0: return 0.0 return bought_both / bought_phone # Historical Flipkart User Data total_phone_buyers = 500 buyers_of_both_phone_and_headphones = 175 p_headphones_given_phone = recommendation_probability( total_phone_buyers, buyers_of_both_phone_and_headphones ) print(f"P(Headphones | Phone) = {p_headphones_given_phone:.2f}")
```

APPLICATION IN RECOMMENDATION ALGORITHMS

Since **P(Headphones | Phone) = 35.0%** exceeds the baseline probability of buying headphones overall (e.g., 5%), Flipkart's engine automatically triggers a targeted recommendation badge ("Frequently Bought Together") on the checkout page to maximize cross-selling conversion.